Leetcode 921 Minimum Add to Make Parenthesis Valid

```python
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        stack = []
        additions = 0

        for ch in s:
            if ch == '(':
                stack.append(ch)
            else:  # ch == ')'
                if stack:
                    stack.pop()
                else:
                    additions += 1  # Need one '('

        # Remaining '(' need ')'
        return additions + len(stack)
```

Brute force
Dry Run

1st Test case
"( ))"

stack = [ ]
additions = 0

Step 1 -> '('
Push to stack

stack = ['(']
additions = 0

Step 2 ')'
stack not empty -> pop
stack = ( )
additions = 0

Step - 3 ')'
addition = 1
stack = [ ]

End
Remaining stack size = 0
Result = additions + len(stack)
= 1 + 0 = 1

Test Case - 2 .. "((("
stack = ( )
additions = 0

Step 1 '('

stack = ['(']

Step 2 → '('

stack = ['(', '(']

Step 3 : '('

stack = ['(', '(', '(']

End,

Remaining '(' = 3

Result = additions + len(stack)

= 0 + 3 = 3

T(→) O(n) S(→) O(n)

```python
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        open_count = 0
        additions = 0

        for ch in s:
            if ch == '(':
                open_count += 1
            else:
                if open_count > 0:
                    open_count -= 1
                else:
                    additions += 1

        return additions + open_count
```

Better Approach

Two Counters

Test - 1 "(()"

open_count = 0

additions = 0

Step - 1 '('

open_count = 1

Step - 2 ')'

open_count > 0 → open_count = open_count - 1

open_count = 0

Step 3 ')'

open_count = 0 → cannot Match

additions = 1

Result = additions + open_count

= 1 + 0 = 1

Test Case - '(((('

Initial : open_count = 0
additions = 0

Step 1 : '('
open_count = 1

Step 2 : '('
open_count = 2

Step 3 : '('
open_count = 3

End,
Result = additions + open_count
= 0 + 3 = 3

T(-) O(n) , S(-) O(1)

```python
class Solution:
    def minAddToMakeValid(self, s: str) -> int:
        balance = 0
        additions = 0

        for ch in s:
            if ch == '(':
                balance += 1
            else:
                balance -= 1

                if balance < 0:
                    additions += 1
                    balance = 0

        return additions + balance
```

Most Optimal

Greedy Balance Method

Test - 1 '())'

Initial, balance = 0
additions = 0

Step 1: '('
balance = 1

Step 2 ')'
balance = 0

Step 3 ')'
balance = -1 ✗ Invalid
addition += 1 → 1
balance reset to 0

End, Result = additions + balance
= 1 + 0 = 1

Dry Run: "((("

Initial : balance = 0
additions = 0

Step 1: '('
balance = 1

Step 2 : '('
balance = 2

Step 3 '('
balance = 3

End, Remaining balance = 3
Result = additions + balance
= 0 + 3 = 3

T(→ O(n)
S(→ O(1)